

the header file generated by Yacc, containing the token definitions, must be included into most of the other code files, but cannot be included directly in the Yacc-generated parser, since that would repeat the internal definitions. The solution is to begin the Yacc declaration section with the definition of a flag **YYPARSER** (line 4007), which will be included in the Yacc parser and will indicate when the C compiler is inside the parser. We use that flag (lines 4226–4236) to selectively include the Yacc-generated header file **y.tab.h** into **globals.h**.

A second problem occurs with the **ENDFILE** token, which the scanner generates to indicate the end of the input file. Yacc assumes this token always has the value 0. We thus provide a direct definition of this token (line 4234) and include it in the selectively compiled section controlled by **YYPARSER**, since it is not needed internally by Yacc.

The final change to the **globals.h** file is to redefine **TokenType** to be a synonym for **int** (line 4252), since the Yacc tokens all have integer values. This avoids unnecessary replacements of the previous enumerated type **TokenType** in other files.

5.7 ERROR RECOVERY IN BOTTOM-UP PARSERS

5.7.1 Detecting Errors in Bottom-Up Parsing

A bottom-up parser will detect an error when a blank (or error) entry is detected in the parsing table. Obviously, it makes sense that errors should be detected as soon as possible, since error messages can be more meaningful and specific. Thus, a parsing table should have as many blank entries as possible.

Unfortunately, this goal conflicts with an equally important one: reducing the size of the parsing table. We have already seen (Table 5.11) that Yacc fills in as many table entries as it can with default reductions, so that a large number of reductions may take place on the parsing stack before an error is declared. This obscures the precise source of the error and may lead to uninformative error messages.

An additional feature of bottom-up parsing is that the power of the particular algorithm used can affect the ability of the parser to detect errors early. An LR(1) parser can, for example, detect errors earlier than an LALR(1) or SLR(1) parser, and these latter can detect errors earlier than an LR(0) parser. As a simple example of this, compare the LR(0) parsing table of Table 5.4 (page 209) with the LR(1) parsing table of Table 5.10 (page 222) for the same grammar. Given the incorrect input string (**a** \$, the LR(1) parsing table of Table 5.10 will shift (and **a** onto the stack and move to state 6. In state 6 there is no entry under \$, so an error will be reported. By contrast, the LR(0) algorithm (and the SLR(1) algorithm as well) will reduce by $A \rightarrow \mathbf{a}$ before discovering the lack of a balancing right parenthesis. Similarly, given the incorrect string (**a**) \$, the general LR(1) parser will shift **a** and then declare error from state 3 on right parenthesis, while the LR(0) parser will again reduce by $A \rightarrow \mathbf{a}$ before declaring error. Of course, any bottom-up parser will always report the error eventually, after perhaps a number of “erroneous” reductions. None of these parsers will ever shift a token on error.

5.7.2 Panic Mode Error Recovery

As in top-down parsing, it is possible to achieve reasonably good error recovery in bottom-up parsers by judiciously removing symbols from either the parsing stack or

the input or both. Similar to LL(1) parsers, there are three possible alternative actions that might be contemplated:

- 1. Pop a state from the stack.
- 2. Successively pop tokens from the input until a token is seen for which we can restart the parse.
- 3. Push a new state onto the stack.

One particularly effective method for choosing which of these actions to take when an error occurs is as follows:

- 1. Pop states from the parsing stack until a state is found with nonempty Goto entries.
- 2. If there is a legal action on the current input token from one of the Goto states, push that state onto the stack and restart the parse. If there are several such states, prefer a shift to a reduce. Among the reduce actions, prefer one whose associated nonterminal is least general.
- 3. If there is no legal action on the current input token from one of the Goto states, advance the input until there is a legal action or the end of the input is reached.

These rules have the effect of forcing the recognition of a construct which was in the process of being recognized when the error occurred, and restarting the parse immediately thereafter. Error recovery that uses these or similar rules could be called **panic mode** error recovery, since it is similar to the top-down panic mode described in Section 4.5.

Unfortunately, these rules can result in an infinite loop, since step 2 pushes new states onto the stack. In that case, there are several possible solutions. One is to insist on a shift action from a Goto state in step 2. This may be too restrictive, however. Another solution is, if the next legal move is a reduction, to set a flag that causes the parser to keep track of the sequence of states during the following reductions, and if the same state recurs, to pop stack states until the original state is removed at which the error occurred, and begin again with step 1. If, at any time, a shift action occurs, the parser resets the flag and continues with a normal parse.

Example 5.19 Consider the simple arithmetic expression grammar whose Yacc parsing table is given in Table 5.11 (page 235). Now consider the erroneous input **(2+*)**. The parse proceeds normally until the ***** is seen. At that point, panic mode would cause the following actions to take place on the parsing stack:

Parsing stack	Input	Action
...	...	...
\$ 0 (6 E 10 + 7	*)\$	error: push T, goto 11
\$ 0 (6 E 10 + 7 T 11	*)\$	shift 9
\$ 0 (6 E 10 + 7 T 11 * 9	)\$	error: push F, goto 13
\$ 0 (6 E 10 + 7 T 11 * 9 F 13	)\$	reduce $T \rightarrow T * F$

At the first error, the parser is in state 7, which has legal Goto states 11 and 4. Since state 11 has a shift on the next input token `*`, that Goto is preferred, and the token is shifted. At that point the parser is in state 9, with a right parenthesis on the input. This is again an error. In state 9 there is a single Goto entry (to state 11), and state 11 does have a legal action on `)` (albeit a reduction). The parse then proceeds normally to its conclusion. §

5.7.3 Error Recovery in Yacc

An alternative to panic mode is to use so-called **error productions**. An error production is a production that contains the pseudotoken **error** on its right-hand side. An error production marks a context in which erroneous tokens can be deleted until appropriate synchronizing tokens are seen, whence the parse can be restarted. In effect, error productions allow the programmer to manually mark those nonterminals whose Goto entries are to be used for error recovery.

Error productions are the principal method available in Yacc for error recovery. The behavior of a Yacc parser in the presence of errors, and its handling of error productions, is as follows.

1. When the parser detects an error during a parse (that is, it encounters an empty entry in the parsing table), it pops states from the parsing stack until it reaches a state in which the **error** pseudotoken is a legal lookahead. The effect is to discard input to the left of the error, and to view the input as containing the **error** pseudotoken. If there are no error productions, then **error** is never a legal lookahead for a shift, and the parser stack will be emptied, aborting the parse at the first error. (This is the behavior of the parser generated by the Yacc input of Figure 5.10, page 228.)
2. Once the parser has found a state on the stack in which **error** is a legal lookahead, it continues in normal fashion with shifts and reduces. The effect is as though **error** were seen in the input, followed by the original lookahead (that is, the lookahead that caused the error). If desired, the Yacc macro **yyclearin** can be used to discard the token that caused the error, and to use the token following it as the next lookahead (after **error**).
3. If, after an error occurs, the parser discovers further errors, then the input tokens causing the errors are silently discarded, until three successive tokens are shifted legally onto the parsing stack. During this time the parser is said to be in an “error state.” This behavior is designed to avoid cascades of error messages caused by the same error. This can, however, result in large amounts of input being thrown away before the parser exits the error state (just as with panic mode). The compiler writer can override this behavior by using the Yacc macro **yyerrok** to remove the parser from the error state, so that no further input will be discarded without a new error recovery.

We describe a few simple examples of this behavior, based on the Yacc input of Figure 5.10.

Example 5.20

Consider the following replacement for the rule for *command* in Figure 5.10:

```

command : exp      { printf("%d\n", $1); }
          | error    { yyerror("incorrect expression"); }
          ;

```

Consider also the erroneous input **2++3**. The parse of this string proceeds normally until the erroneous second **+** is reached. At that point the parse is given by the following parsing stack and input (we use Table 5.11 to describe this, although the addition of the error production will actually result in a slightly different parse table):

PARSING STACK	INPUT
\$0 <i>exp</i> 2 + 7	+3\$

Now the parser enters the error “state” (generating an error message such as “syntax error”), and begins popping states from the stack, until state 0 is uncovered. At this point the error production for *command* provides that **error** is a legal lookahead, and it will be shifted onto the parsing stack, and immediately reduced to *command*, causing the associated action to be executed (which prints the message “incorrect expression”). The resulting situation is now as follows:

PARSING STACK	INPUT
\$0 <i>command</i> 1	+3\$

At this point the only legal lookahead is the end of input (indicated by \$ here, corresponding to the return of 0 by **yylex**), and the parser will delete the remaining input tokens **+3** before exiting (while still in the “error state”). Thus, the addition of the error production has essentially the same effect as the version in Figure 5.10, except that we are now able to supply our own error message.

An even better error mechanism than this would allow the user to reenter the line after the erroneous input. In this case, a synchronizing token is needed, and the end of line marker is the only sensible one. Thus, the scanner must be modified to return the newline character (instead of 0), and with this modification we may write (but see Exercise 5.32):

```

command : exp '\n'    { printf("%d\n", $1); exit(0); }
          | error '\n'
            { yyerrok;
              printf("reenter expression: "); }
          command
          ;

```

This code has the effect that, when an error occurs, the parser will skip all tokens up to a newline, when it will execute the action represented by **yyerrok** and the **printf** statement. Then it will attempt to recognize another *command*. The call to **yyerrok** is needed here to cancel the “error state” after the newline is seen, since otherwise if a new

error occurs right after the newline, Yacc will silently delete tokens until it finds a sequence of three correct tokens. §

Example 5.21

Consider what would happen if an error production were added to the Yacc definition of Figure 5.10 as follows:

```

factor      : NUMBER          { $$ = $1; }
              | '(' exp ')'      { $$ = $2; }
              | error { $$ = 0; }
              ;

```

Consider first the erroneous input **2++3** as in the previous example. (We continue to use Table 5.11, although the additional error production results in a slightly different table.) As before, the parser will reach the following point:

PARSING STACK	INPUT
\$0 <i>exp</i> 2 + 7	+3\$

Now the error production for *factor* will provide that **error** is a legal lookahead in state 7 and **error** will be immediately shifted onto the stack and reduced to *factor*, causing the value 0 to be returned. Now the parser has reached the following point:

PARSING STACK	INPUT
\$0 <i>exp</i> 2 + 7 <i>factor</i> 4	+3\$

This is a normal situation, and the parser will continue to execute normally to the end. The effect is to interpret the input as **2+0+3**—the 0 between the two + symbols is there because that is where the **error** pseudotoken is inserted, and by the action for the error production, **error** is viewed as equivalent to a factor with value 0.

Now consider the erroneous input **2 3** (that is, two numbers with a missing operator). Here the parser reaches the position

PARSING STACK	INPUT
\$0 <i>exp</i> 2	3\$

At this point (if the rule for *command* has not been changed), the parser will (erroneously) reduce by the rule *command* → *exp* (and print the value 2), even though a number is not a legal follow symbol for *command*. The parser then reaches the position

PARSING STACK	INPUT
\$0 <i>command</i> 1	3\$

Now an error is detected, and state 1 is popped from the parsing stack, uncovering state 0. At this point the error production for *factor* allows for **error** to be a legal lookahead from state 0, and **error** is shifted onto the parsing stack, resulting in another cascade of reductions and the printing of the value 0 (the value returned by the error production). Now the parser is back at the same position in the parse, with the number **3** still

in the input! Fortunately, the parser is already in the “error state” and will not shift **error** again, but throws away the number **3**, uncovering the correct lookahead for state 1, whereupon the parser exits.¹⁰ The result is that the parser prints the following (we repeat the user input on the first line):

```
> 2 3
2
syntax error
incorrect expression
0
```

This behavior gives a glimpse of the difficulties of good error recovery in Yacc. (See the exercises for more examples.) §

5.7.4 Error Recovery in TINY

The Yacc specification file **tiny.y** in Appendix B contains two error productions, one for **stmt** (line 4047) and one for **factor** (line 4139), whose associated actions are to return the null syntax tree. These error productions provide a level of error handling similar to that of the recursive-descent TINY parser in the previous chapter, except that no attempt is made to build a significant syntax tree in the presence of errors. Also, these error productions provide no special synchronization for the restart of the parse, so in the presence of multiple errors many tokens may be skipped.

EXERCISES

5.1 Consider the following grammar:

$$\begin{aligned} E &\rightarrow (L) \mid a \\ L &\rightarrow L , E \mid E \end{aligned}$$

- a. Construct the DFA of LR(0) items for this grammar.
 - b. Construct the SLR(1) parsing table.
 - c. Show the parsing stack and the actions of an SLR(1) parser for the input string $((a), a, (a, a))$.
 - d. Is this grammar an LR(0) grammar? If not, describe the LR(0) conflict. If so, construct the LR(0) parsing table, and describe how a parse might differ from an SLR(1) parse.
- 5.2 Consider the grammar of the previous exercise.
- a. Construct the DFA of LR(1) items for this grammar.
 - b. Construct the general LR(1) parsing table.
 - c. Construct the DFA of LALR(1) items for this grammar.
 - d. Construct the LALR(1) parsing table.

10. Some versions of Yacc pop the parsing stack again before deleting any input. This can result in even more complicated behavior. See the exercises.